

easy-db Shell 工具开发指导书

一、模块概述

1.1 什么是 Shell 工具？

Shell 工具是指将数据库操作命令集成到操作系统的 Shell（命令行终端）中，让用户可以直接在系统的 Bash/Zsh 终端中调用数据库功能，而不需要启动一个专用的交互式程序。

与 CLI 工具的区别：

对比项	CLI 工具	Shell 工具
运行方式	启动一个 Java 程序，进入交互循环	直接在终端执行单条命令，立即返回
使用场景	人工交互式操作	脚本调用、管道组合、自动化任务
典型调用	<code>java -jar cli.jar</code> 然后输入命令	<code>easy-db SET name 张三</code> 直接执行
进程生命周期	长期运行	每次执行完即退出

通俗理解：

- CLI 工具 = 进入了数据库的“专属聊天室”，在里面不断输入命令
- Shell 工具 = 把数据库命令做成了“终端快捷指令”，像 `ls`、`grep` 一样随时调用

1.2 效果预览

```
# 直接执行单条命令（类似 ls、grep 的风格）
$ easy-db set name 张三
OK

$ easy-db get name
"张三"

# 配合管道使用
$ echo "李四" | easy-db set name2
OK

# 配合其他 Shell 命令组合
$ easy-db keys user:* | while read key; do easy-db get $key; done
"张三"
"李四"
"王五"
```

二、功能需求

2.1 核心功能

目标：提供一组 Shell 命令函数，用户可以直接在终端中调用。

需求 2.1.1：命令函数定义

定义以下 Shell 函数（以 Bash 为例）：

函数名	对应数据库命令	说明
<code>easy-db-set</code>	<code>SET key value</code>	存储键值对
<code>easy-db-get</code>	<code>GET key</code>	获取值
<code>easy-db-del</code>	<code>DEL key</code>	删除键值对
<code>easy-db-keys</code>	<code>KEYS [pattern]</code>	列出所有键
<code>easy-db-exists</code>	<code>EXISTS key</code>	检查键是否存在
<code>easy-db-flush</code>	<code>FLUSH</code>	清空所有数据

需求 2.1.2：统一入口（推荐）

除了独立的函数，还提供一个**统一入口脚本**，通过子命令调用：

```
# 统一入口方式（类似 git 的风格）
$ easy-db set name 张三
$ easy-db get name
$ easy-db keys user:*
```

这样更简洁，也更容易扩展。

需求 2.1.3：连接配置

支持通过**环境变量**或**配置文件**指定服务器地址和端口，避免每次都要输入。

环境变量	默认值	说明
<code>EASY_DB_HOST</code>	<code>127.0.0.1</code>	服务器地址
<code>EASY_DB_PORT</code>	<code>8080</code>	服务器端口

使用示例：

```
# 使用默认连接
$ easy-db get name

# 临时切换连接（通过环境变量）
$ EASY_DB_HOST=192.168.1.100 easy-db get name

# 永久配置（写入 ~/.bashrc）
$ export EASY_DB_HOST=192.168.1.100
$ export EASY_DB_PORT=9090
```

2.2 输出格式

需求 2.2.1：成功输出

```
$ easy-db set name 张三
OK
```

需求 2.2.2：查询输出

```
# 普通值直接输出（方便管道处理）
$ easy-db get name
张三

# 多个值按行输出
$ easy-db keys *
user:001
user:002
user:003
```

需求 2.2.3：错误输出

```
$ easy-db get notexist
(nil)

$ easy-db set
(error) ERR wrong number of arguments for 'set' command
```

需求 2.2.4：静默模式（供脚本调用）

添加 `-s` 或 `--silent` 选项，只输出数据，不输出额外的提示信息（方便脚本解析）。

```
$ easy-db -s get name
张三
```

2.3 增强功能（选做加分）

功能	说明	加分
JSON 格式化输出	查询结果如果是 JSON，用 <code>jq</code> 风格格式化	+3
批量操作	支持 <code>--file</code> 参数批量执行命令文件	+3
交互式补全	在 Shell 中按 Tab 自动补全命令和 key	+4
彩色输出	不同类型的输出使用不同颜色	+2

三、实现方案

3.1 方案 A：纯 Shell 脚本实现（仅供参考，不需要实现）

优点：无需额外编译，直接修改即可运行，简单直观。

缺点：功能相对有限，复杂逻辑较难实现。

实现步骤：

第一步：创建脚本文件

在项目根目录创建 `easy-db-shell.sh`：

```
#!/bin/bash

# =====
# easy-db Shell 工具
# 用法：easy-db <command> [args...]
# 示例：easy-db set name 张三
```

```

# =====

# 配置（可通过环境变量覆盖）
EASY_DB_HOST="${EASY_DB_HOST:-127.0.0.1}"
EASY_DB_PORT="${EASY_DB_PORT:-8080}"

# 颜色定义
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[0;33m'
NC='\033[0m' # No Color

# =====
# 核心函数：发送命令到服务端
# =====

easy_db_send() {
    local command="$1"

    # 使用 nc (netcat) 发送命令并接收响应
    # 也可以用 telnet、curl 等方式
    echo "$command" | nc -w 2 "$EASY_DB_HOST" "$EASY_DB_PORT" 2>/dev/null

    # 检查连接是否成功
    if [ $? -ne 0 ]; then
        echo -e "${RED}(error) Failed to connect to $EASY_DB_HOST:$EASY_DB_PORT${NC}" >&2
        return 1
    fi
}

# =====
# 命令函数
# =====

# SET key value
easy_db_set() {
    if [ $# -lt 2 ]; then
        echo -e "${RED}(error) Usage: set <key> <value>${NC}" >&2
        return 1
    fi
    local key="$1"
    local value="$2"
    easy_db_send "SET $key $value"
}

# GET key
easy_db_get() {
    if [ $# -lt 1 ]; then
        echo -e "${RED}(error) Usage: get <key>${NC}" >&2
        return 1
    fi
    easy_db_send "GET $1"
}

# DEL key
easy_db_del() {
    if [ $# -lt 1 ]; then
        echo -e "${RED}(error) Usage: del <key>${NC}" >&2
        return 1
    fi
    easy_db_send "DEL $1"
}

```

```

    fi
    easy_db_send "DEL $1"
}

# KEYS [pattern]
easy_db_keys() {
    local pattern="${1:-*}"
    easy_db_send "KEYS $pattern"
}

# EXISTS key
easy_db_exists() {
    if [ $# -lt 1 ]; then
        echo -e "${RED}(error) Usage: exists <key>${NC}" >&2
        return 1
    fi
    easy_db_send "EXISTS $1"
}

# FLUSH
easy_db_flush() {
    easy_db_send "FLUSH"
}

# =====
# 帮助信息
# =====
easy_db_help() {
    cat << EOF
easy-db Shell Tool - 命令行数据库客户端

用法：
    easy-db <command> [args...]

命令：
    set <key> <value>      存储键值对
    get <key>              获取键对应的值
    del <key>              删除键值对
    keys [pattern]         列出所有键
    exists <key>           检查键是否存在
    flush                  清空所有数据
    help                   显示帮助信息

环境变量：
    EASY_DB_HOST           服务器地址 (默认: 127.0.0.1)
    EASY_DB_PORT           服务器端口 (默认: 8080)

示例：
    easy-db set name 张三
    easy-db get name
    easy-db keys user:*

EOF
}

# =====
# 主入口
# =====

```

```

easy_db_main() {
    local cmd="$1"
    shift

    case "$cmd" in
        set)
            easy_db_set "$@"
            ;;
        get)
            easy_db_get "$@"
            ;;
        del)
            easy_db_del "$@"
            ;;
        keys)
            easy_db_keys "$@"
            ;;
        exists)
            easy_db_exists "$@"
            ;;
        flush)
            easy_db_flush "$@"
            ;;
        help|--help|-h)
            easy_db_help
            ;;
        "")
            easy_db_help
            ;;
        *)
            echo -e "${RED}(error) Unknown command: $cmd${NC}" >&2
            echo "Run 'easy-db help' for usage." >&2
            return 1
            ;;
    esac
}

# 如果脚本被直接执行（而不是被 source）
if [ "${BASH_SOURCE[0]}" = "${0}" ]; then
    easy_db_main "$@"
fi

```

第二步：使用方法

```

# 1. 给脚本添加执行权限
$ chmod +x easy-db-shell.sh

# 2. 直接执行（需要指定完整路径）
$ ./easy-db-shell.sh set name 张三
OK

# 3. 或者将其添加到 PATH 中（推荐）
$ sudo cp easy-db-shell.sh /usr/local/bin/easy-db
$ easy-db get name
"张三"

# 4. 如果希望命令在子 shell 中也能使用，可以 source 脚本
$ source easy-db-shell.sh

```

```
# 然后就可以直接调用内部函数了
```

```
$ easy_db_get name
```

3.2 方案 B: Java 实现 + Shell 包装（需要实现）

优点：功能更强大，可以复用已有的 `SocketClient` 代码。

缺点：需要编译打包。

第一步：创建 Java 入口类

在 `client` 包下创建 `ShellClient.java`：

```
package client;

public class ShellClient {
    private static String host = "127.0.0.1";
    private static int port = 8080;
    private static boolean silent = false;

    public static void main(String[] args) {
        // 解析环境变量
        host = System.getenv().getOrDefault("EASY_DB_HOST", host);
        String portStr = System.getenv().getOrDefault("EASY_DB_PORT", String.valueOf(port));
        port = Integer.parseInt(portStr);

        // 解析命令行参数
        if (args.length == 0) {
            printHelp();
            return;
        }

        // 解析全局选项
        List<String> commandArgs = new ArrayList<>();
        for (String arg : args) {
            if (arg.equals("-s") || arg.equals("--silent")) {
                silent = true;
            } else if (arg.equals("-h") || arg.equals("--help")) {
                printHelp();
                return;
            } else {
                commandArgs.add(arg);
            }
        }

        if (commandArgs.isEmpty()) {
            printHelp();
            return;
        }

        String cmd = commandArgs.get(0);
        List<String> params = commandArgs.subList(1, commandArgs.size());

        // 执行命令
        try {
            SocketClient client = new SocketClient(host, port);
            client.connect();

            String result = executeCommand(client, cmd, params);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static void printHelp() {
        System.out.println("Usage: easy_db_get [-s] [-h] [command] [args...]\n");
        System.out.println("-s, --silent: Silent mode\n");
        System.out.println("-h, --help: Show this help message\n");
    }
}
```

```

        if (result != null) {
            System.out.println(result);
        }

        client.disconnect();
    } catch (Exception e) {
        if (!silent) {
            System.err.println("(error) " + e.getMessage());
        }
        System.exit(1);
    }
}

private static String executeCommand(SocketClient client, String cmd, List<String> params) {
    // 构建完整命令
    StringBuilder fullCmd = new StringBuilder(cmd.toUpperCase());
    for (String p : params) {
        fullCmd.append(" ").append(p);
    }
    return client.sendCommand(fullCmd.toString());
}

private static void printHelp() {
    System.out.println("easy-db Shell Tool (Java)");
    System.out.println("");
    System.out.println("Usage: java -jar easy-db-shell.jar <command> [args...]");
    System.out.println("");
    System.out.println("Environment variables:");
    System.out.println("  EASY_DB_HOST    Server host (default: 127.0.0.1)");
    System.out.println("  EASY_DB_PORT    Server port (default: 8080)");
    // ... 更多帮助信息
}
}

```

第二步：创建 Shell 包装脚本

创建一个名为 `easy-db` 的 Shell 脚本（无扩展名），放在项目根目录：

```

#!/bin/bash
# easy-db Shell 包装脚本

# 找到 jar 包的位置（假设和脚本在同一个目录）
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
JAR_PATH="$SCRIPT_DIR/target/easy-db-shell.jar"

# 如果 jar 包不存在，尝试编译
if [ ! -f "$JAR_PATH" ]; then
    echo "Building easy-db-shell..." >&2
    cd "$SCRIPT_DIR"
    mvn clean package -DskipTests >/dev/null 2>&1
fi

# 执行 Java 程序，传递所有参数
java -jar "$JAR_PATH" "$@"

```

第三步：使用方法


```
# 编译打包
$ mvn clean package

# 直接使用（通过包装脚本）
$ ./easy-db set name 张三
OK

# 将脚本添加到 PATH
$ sudo ln -s $(pwd)/easy-db /usr/local/bin/easy-db
$ easy-db get name
"张三"

# 配合管道使用
$ echo "王五" | xargs easy-db set name3
OK
```

3.3 方案选择建议

方案	适合学生	优点	缺点
方案 A（纯 Shell）	大二学生，Shell 基础一般	简单直观、无需编译、跨平台易用	依赖 nc 命令、功能受限
方案 B（Java+包装）	已掌握 Java 打包的学生	可复用现有代码、功能更强大	需要编译和打包

四、实际使用示例

4.1 基础操作

```
# 存储数据
$ easy-db set user:001 张三
OK
$ easy-db set user:002 李四
OK
$ easy-db set user:003 "王 五" # 值中包含空格需要用引号
OK

# 查询数据
$ easy-db get user:001
"张三"

# 列出所有键
$ easy-db keys *
user:001
user:002
user:003

# 列出匹配的键
$ easy-db keys user:00*
user:001
user:002
user:003

# 检查存在
$ easy-db exists user:001
true
```

```
# 删除数据
$ easy-db del user:002
OK
$ easy-db get user:002
(nil)
```

4.2 组合使用 (Shell 管道)

这是 Shell 工具的最大优势——可以和其他 Shell 命令组合使用：

```
# 导出所有数据
$ easy-db keys * | while read key; do
    value=$(easy-db get "$key")
    echo "$key -> $value"
done
user:001 -> "张三"
user:003 -> "王 五"

# 备份所有数据到文件
$ easy-db keys * | while read key; do
    value=$(easy-db get "$key")
    echo "$key $value"
done > backup.txt

# 从备份文件恢复
$ cat backup.txt | while read key value; do
    easy-db set "$key" "$value"
done

# 统计数据量
$ easy-db keys * | wc -l
10

# 查找包含特定值的键
$ easy-db keys * | while read key; do
    if easy-db get "$key" | grep -q "三"; then
        echo "$key"
    fi
done
user:001

# 清空所有数据
$ easy-db flush
OK
```

4.3 在脚本中使用

创建 `backup.sh` 脚本：

```
#!/bin/bash
# 数据库备份脚本

BACKUP_FILE="backup_$(date +%Y%m%d_%H%M%S).txt"

echo "开始备份数据库..."

easy-db keys * | while read key; do
```

```
value=$(easy-db get "$key")
echo "SET $key $value" >> "$BACKUP_FILE"
done

echo "备份完成: $BACKUP_FILE"
echo "共有 $(wc -l < $BACKUP_FILE) 条记录"
```

运行效果:

```
$ ./backup.sh
开始备份数据库...
备份完成: backup_20240618_143022.txt
共有 15 条记录
```

4.4 配置持久化 (Bash 用户)

将以下内容添加到 `~/.bashrc` 或 `~/.zshrc` :

```
# easy-db 配置
export EASY_DB_HOST=127.0.0.1
export EASY_DB_PORT=8080

# 如果 easy-db 命令在自定义路径, 可以添加 alias
alias easy-db='/path/to/easy-db'

# 添加快捷函数
function db() {
    easy-db "$@"
}
```

之后就可以直接使用 `db` 作为快捷命令:

```
$ db set name 张三
OK
$ db get name
"张三"
```

五、开发任务清单

阶段	任务	状态
阶段一	创建 <code>easy-db-shell.sh</code> 脚本文件	☐
阶段二	实现 <code>easy_db_send()</code> 核心通信函数	☐
阶段三	实现 <code>set / get / del</code> 三个核心命令	☐
阶段四	实现 <code>keys / exists / flush</code> 命令	☐
阶段五	实现 <code>help</code> 命令和参数解析	☐
阶段六	配置环境变量支持 (<code>EASY_DB_HOST / PORT</code>)	☐
阶段七	添加错误处理和友好提示	☐
阶段八 (选做)	JSON 格式化输出	☐
阶段九 (选做)	Tab 自动补全支持	☐

七、常见问题

Q1: `nc` 命令不存在怎么办？

A: macOS/Linux 一般自带 `nc` (netcat) 。如果没有，可以用以下方式替代：

```
# 方式1: 使用 telnet
echo "$command" | telnet "$EASY_DB_HOST" "$EASY_DB_PORT" 2>/dev/null

# 方式2: 使用 /dev/tcp (仅 Bash)
exec 3<>/dev/tcp/$EASY_DB_HOST/$EASY_DB_PORT
echo "$command" >&3
cat <&3
exec 3<&-
```

Q2: 服务器返回的数据包含中文，Shell 显示乱码？

A: 确保终端和脚本都使用 UTF-8 编码：

```
export LANG=en_US.UTF-8
export LC_ALL=en_US.UTF-8
```

Q3: 如何在 macOS 上将脚本添加到 PATH？

A:

```
# 方法1: 创建软链接
sudo ln -s /path/to/easy-db-shell.sh /usr/local/bin/easy-db

# 方法2: 添加到 ~/.zshrc (macOS 默认使用 zsh)
echo 'export PATH=$PATH:/path/to/script/dir' >> ~/.zshrc
source ~/.zshrc
```

Q4: 如何处理带空格的 value？

A: 在 Shell 中使用引号包裹：

```
easy-db set name "张 三"
easy-db get name # 输出: "张 三"
```

Q5: 如何实现 Tab 自动补全?

A: 需要编写 Bash 补全脚本, 放在 `/etc/bash_completion.d/` 或 `/usr/local/etc/bash_completion.d/`:

```
# easy-db 补全脚本
_easy_db_completion() {
    local cur="${COMP_WORDS[COMP_CWORD]}"
    local commands="set get del keys exists flush help"

    if [ $COMP_CWORD -eq 1 ]; then
        COMPREPLY=( $(compgen -W "$commands" -- "$cur") )
    fi
}
complete -F _easy_db_completion easy-db
```

七、提交要求

1. 代码: Shell 脚本文件 `easy-db-shell.sh` (或 Java 实现 + 打包)
2. 文档: 在课程设计报告中增加 **Shell 工具** 章节 (1-2 页), 内容包括:
 - 设计思路
 - 核心代码说明
 - 使用示例截图 (至少 5 张)
 - 遇到的问题及解决
3. 演示: 验收时现场演示 Shell 工具的各种用法, 包括管道组合

八、参考资料

- [Bash 脚本编程指南](#)
- [netcat\(nc\) 命令用法](#)
- [Bash 补全 \(Tab 自动补全\)](#)
- [环境变量在 Shell 中的使用](#)

祝同学们顺利完成 Shell 工具开发!